

Fiches de Cours

Master IMA

Université de Toulouse III, Paul Sabatier
Années universitaires 2025-2027

22 septembre 2025

Table des matières

I Probabilités	3
1 Chaînes de Markov	4
1.1 Définitions et Généralités	4
1.2 Propriétés de Markov	9
II Simulation Aléatoire	12
1 Simulation de variables aléatoires	13
1.1 Simulation de variables aléatoires discrètes	14
III Optimisation	17
1 Optimisation sans contraintes	18
1.1 CNS d'optimalité dans contraintes	18
1.2 Algorithmes de Descente	20
IV Algorithmique	23
1 Flot maximal dans un graphe	24
1.1 Rappels sur les graphes	24
1.2 Cycle et flot	26
1.3 Flot et coupure	27
1.4 Flot maximal : principe de Ford-Fulkerson	29
V Compilation	30
1 Analyse Syntaxique	31
1.1 Analyse Syntaxique et approche descendante (Grammaires $LL(k)$)	31
1.2 Analyse Syntaxique et approche ascendante (Grammaires $LR(k)$)	35

Probabilités

Chapitre 1

Chaînes de Markov

Contents

1.1 Définitions et Généralités	4
1.2 Propriétés de Markov	9

Jusqu'à maintenant, nous avons étudié des suites de variables aléatoires dites iid (indépendantes et identiquement distribuées). Ces propriétés nous ont permis de développer beaucoup d'outils pour les étudier tels que la loi forte des grands nombres ou le théorème central limite.

Or nous n'utilisons pas beaucoup ce type de suite de variables aléatoires en simulation. En effet, ces propriétés plutôt fortes ne permettent pas de modéliser des phénomènes dont l'évolution est plus complexe. Nous allons donc continuer à étudier des suites de variables aléatoires mais en rajoutant des relations de dépendance entre celles-ci. Cela nous permettra de modéliser des systèmes dynamiques au cours du temps plus complexes.

1.1 Définitions et Généralités

On se place ici dans un espace probabilisé $(E, (F), \mathbb{P})$ et E un *espace d'état* fini ou dénombrable. On s'intéresse à une suite de variables aléatoires $(X_n)_{n \in \mathbb{N}}$ ou X_k est l'état du système au temps k .

Définition (Trajectoire) . On appelle *trajectoire du système* jusqu'au temps $n \in \mathbb{N}^*$ la donnée $(X_1, \dots, X_n)$.

Définition (Chaîne de Markov) . On dit qu'une suite $(X_n)_{n \geq 0}$ est une *chaîne de Markov* si elle vérifie la *propriété de Markov* :

$$\forall n \in \mathbb{N}, \forall i_0, i_1, \dots, i_{n-1} \in E,$$

$$\mathbb{P}(X_{n+1} = j \mid X_n = i, X_{n-1} = i_{n-1}, \dots, X_0 = i_0) = \mathbb{P}(X_{n+1} = j \mid X_n = i)$$

Remarque Conditionnellement à l'état présent $\{X_n = i\}$, la probabilité d'arriver à l'état $\{X_{n+1} = j\}$ à l'itération suivante ne dépend pas des événements du passé $\{X_0 = i_0, \dots, X_{n-1} = i_{n-1}\}$. On parle d'*absence de mémoire*.

Définition (Chaîne de Markov Homogène) . Soit $(X_n)_{n \geq 0}$ une chaîne de Markov. On dit qu'elle est *homogène* si :

$$\forall n \in \mathbb{N}, \forall i, j \in E, \quad \mathbb{P}(X_{n+1} = j \mid X_n = i) = \mathbb{P}(X_1 = j \mid X_0 = i)$$

On se placera toujours dans ce cadre.

Remarque Une chaîne de Markov homogène est une chaîne de Markov où le temps (n) n'apparaît pas dans la probabilité de passer d'un état à l'autre.

Exemple (Marche Aléatoire) On modélise le mouvement d'une particule sur une grille par une chaîne de Markov où X_n représente la position de la particule au temps $n \in \mathbb{N}$. On considère que la particule se déplace sur une seule dimension. Elle peut donc avancer d'une case ou reculer d'une case à chaque itération. On notera $p \in [0, 1]$ la probabilité qu'elle avance et $1 - p$ la probabilité qu'elle recule indépendamment du passé. Soit $(\varepsilon_i)_{i \geq 0}$ à valeurs dans $\{-1, 1\}$ telle que :

$$\mathbb{P}(\varepsilon_i = 1)1 - \mathbb{P}(\varepsilon_i = -1) = p \in [0, 1]$$

la variable aléatoire qui représente le mouvement de la particule. Déterminons la loi de X_n :

$$X_{n+1} = X_n + \varepsilon_{n+1} = X_{n-1} + \varepsilon_n + \varepsilon_{n+1} = \dots = \sum_{i=1}^n \varepsilon_i$$

Montrons que c'est bien une chaîne de Markov :

$$\begin{aligned} X_{n+1} &= X_n + \varepsilon_{n+1} \\ &= \left(\sum_{i=1}^n \varepsilon_i \right) + \varepsilon_{n+1} \\ &= \sum_{i=1}^{n+1} \varepsilon_i \end{aligned}$$

où tous les $\varepsilon_i, i \in \llbracket 1, n+1 \rrbracket$ sont indépendants. On a donc :

$$\begin{aligned} \mathbb{P}(X_{n+1} = j \mid X_n = i, \dots, X_0 = i_0) &= \mathbb{P}(X_n + \varepsilon_{n+1} = j \mid X_n = i, \dots, X_0 = i_0) \\ &= \mathbb{P}(i + \varepsilon_{n+1} = j \mid X_n = i, \dots, X_0 = i_0) \\ &= \mathbb{P}(\varepsilon_{n+1} = j - i) \end{aligned}$$

On pourra alors s'intéresser au nombre de fois où une trajectoire passe par un point $n \in \mathbb{N}$. La théorie des chaînes de Markov permet de répondre à ce genre de questions.

Plus généralement, on peut construire récursivement des chaînes de Markov en définissant un X_0 fixé ou tiré par une loi π_0 puis en posant :

$$X_{n+1} = f(X_n, \varepsilon_{n+1})$$

où (ε_i) est une suite de variables aléatoires iid et f une fonction mesurable.

Définition (Matrice de Transition) . Soit $(X_n)_{n \geq 0}$ une chaîne de Markov. On définit la *matrice de transition* P de la chaîne comme la matrice, potentiellement infinie, telle que :

$$P(i, j) = \mathbb{P}(X_1 = j \mid X_0 = i)$$

Ainsi le coefficient de la i -ème ligne et de la j -ème colonne correspond à la probabilité de passer de l'état i à l'état j en une étape.

Exemple Soit $P \in \mathcal{M}_3(\mathbb{R})$ la matrice de transition suivante :

$$P = \begin{pmatrix} 1/4 & 3/4 & 0 \\ 0 & 1/2 & 1/2 \\ 1/2 & 0 & 1/2 \end{pmatrix}$$

On peut alors la représenter par un graphe pondéré orienté dont les noeuds représentent les états au temps i de la chaîne de Markov et les poids les probabilités de passer d'un état à l'autre en une étape.

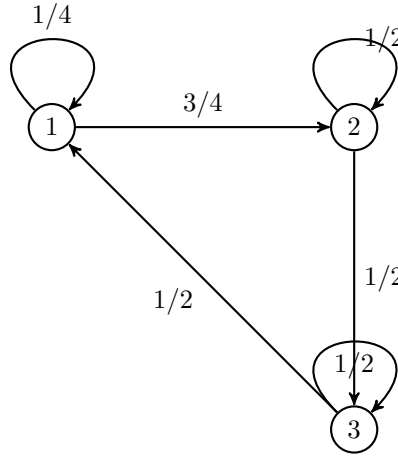


FIGURE 1.1 – Graphe Orienté Pondéré de la matrice de transition P

Définition (Matrice Stochastique) . La matrice de transition P est dite *stochastique* si

$$\forall i \in E, \quad \sum_{j \in E} P(i, j) = 1$$

(i.e si la somme des poids des arrêtes de tout noeud du graphe associé est égale à 1)

Proposition La loi d'une chaîne de Markov homogène $(X_n)_{n \geq 0}$ est complètement déterminée par la loi de π_0 de la condition initiale X_0 et par la matrice de transition P . On a :

$$\forall i_0, i_1, \dots, i_n \in E, \quad \mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n) = \pi_0(i_0)P(i_0, i_1)P(i_1, i_2) \dots P(i_{n-1}, i_n)$$

Démonstration Démontrons ce résultat par récurrence sur $\mathbb{N}$.

- Initialisation : Pour $n = 0$, par définition, on a :

$$\mathbb{P}(X_0 = i_0) = \pi_0(i_0)$$

- Hérédité : Soient $i_0, \dots, i_n \in E$, supposons que :

$$\mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n) = \pi_0(i_0)P(i_0, i_1)P(i_1, i_2) \dots P(i_{n-1}, i_n)$$

Soit $i_{n+1} \in E$ alors :

$$\begin{aligned} & \mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n, X_{n+1} = i_{n+1}) \\ &= \mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n) \times \mathbb{P}(X_{n+1} = i_{n+1} | X_0 = i_0, \dots, X_n = i_n) \\ &= \mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n) \times \mathbb{P}(X_{n+1} = i_{n+1} | X_n = i_n) \\ &= \pi_0(i_0)P(i_0, i_1)P(i_1, i_2) \dots P(i_{n-1}, i_n)P(i_n, i_{n+1}) \end{aligned}$$

On s'intéresse maintenant à la loi de X_n au temps $n \in \mathbb{N}$.

Propriété (Relation de Chapman-Kolmogorov) . Soit $(X_n)_{n \geq 0}$ une chaîne de markov homogène de matrice P et de loi initiale π_0 . Pour tout $i, j \in E$, on a :

$$\mathbb{P}(X_{n+m} = j | X_0 = i_0) = \sum_{k \in E} \mathbb{P}(X_{n+m} = j | X_n = k) \times \mathbb{P}(X_n = k | X_0 = i_0)$$

En particulier :

$$\mathbb{P}(X_n = j | X_0 = i_0) = P^n(i_0, j)$$

Démonstration Prouvons les résultats précédents.

Rappelons tout d'abord la formule des probabilités totales. Soit $(B_k)_{k \in \mathbb{N}}$ une partition de E , alors pour tout $A \in E$, on a :

$$\mathbb{P}(A) = \sum_{k \in \mathbb{N}} \mathbb{P}(A \cap B_k)$$

ou en conditionnel sur C :

$$\mathbb{P}(A|C) = \sum_{k \in \mathbb{N}} \mathbb{P}(A \cap B_k|C)$$

Or ici, $\{X_k | k \geq 0\}$ forme bien une partition de E puisque à chaque instant k , le processus est dans un unique état X_k . D'après la formule des probabilités totales, on a donc :

$$\begin{aligned} \mathbb{P}(X_{n+m} = j | X_0 = i_0) &= \sum_{k \in E} \mathbb{P}(X_{n+m} = j, X_n = k | X_0 = i_0) \\ &= \sum_{k \in E} \mathbb{P}(X_{n+m} = j | X_n = k, X_0 = i_0) \times \mathbb{P}(X_n = k | X_0 = i_0) \\ &= \sum_{k \in E} \mathbb{P}(X_{n+m} = j | X_n = k) \mathbb{P}(X_n = k | X_0 = i_0) \end{aligned}$$

On remarque de $\mathbb{P}(X_{n+m} = j | X_n = k)$ est la probabilité de passer de l'état k à j en m pas d'où :

$$\mathbb{P}(X_{n+m} = j | X_n = k) = \mathbb{P}(X_m = j | X_0 = k)$$

Déduisons maintenant la seconde formule de la première :

$$\begin{aligned} \mathbb{P}(X_2 = j | X_0 = i_0) &= \sum_{k \in E} \mathbb{P}(X_2 = j | X_1 = k) \mathbb{P}(X_1 = k | X_0 = i_0) \\ &= \sum_{k \in E} P(i_0, k)P(k, j) \\ &= P^2(i_0, j) \quad \text{etc} \dots \end{aligned}$$

Remarque (Notation) Si X_0 est distribué selon la loi π_0 , on notera :

$$\begin{aligned}\mathbb{P}_{\pi_0}(X_n = j) &= \sum_{i_0 \in E} \mathbb{P}(X_n = j | X_0 = i_0) \pi_0(i_0) \\ &= \sum_{i_0 \in E} \mathbb{P}(X_n = j | X_0 = i_0) \mathbb{P}(X_0 = i_0) \\ &= \sum_{i_0 \in E} P(i_0, j) \pi_0(i_0) \\ &= \pi_0 P^n(j)\end{aligned}$$

où $P^n(j)$ est la i ème colonne de P^n et π_0 un vecteur ligne.

Exemple (Simple) Soit $(X_n)_{n \geq 0}$ une suite de variables aléatoires réelles iid de loi π .

$$\text{i.e } \forall i \in E, \mathbb{P}(X_n = i) = \pi(i)$$

On a donc :

$$P(i, j) = \mathbb{P}(X_1 = j | X_0 = i) = \mathbb{P}(X_1 = j) = \pi(j)$$

C'est un cas très simple et peu intéressant.

Exemple (Ruine du joueur) Soit $n \geq 1$ et $p \in [0, 1]$ et une suite de variables aléatoires $(X_n)_{n \in \mathbb{N}}$ à valeurs dans $\llbracket 1, n \rrbracket$. On définit X_m comme le montant gagné par le joueur après m parties. On considère que le joueur ne peut pas jouer avec un montant supérieur à n . On peut donc représenter son évolution dans le jeu par le graphe ci-dessous :

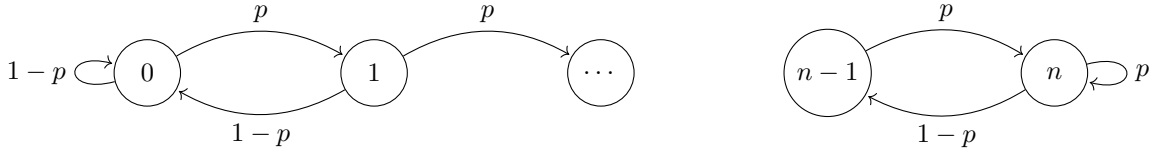


FIGURE 1.2 – Ruine du Joueur

où :

$$\begin{cases} P(k, k+1) = p & \text{si } k \in \{0, \dots, n-1\} \\ P(k, k-1) = 1-p & \text{si } k \in \{1, \dots, n\} \end{cases}$$

En pratique, on s'intéressera souvent à :

$$\mathbb{P}(X_n \text{ atteint } 0 \text{ avant } m | X_0 = i)$$

Exemple (File d'attente) On s'intéresse au nombre de personnes dans une file d'attente où, à chaque itération, une personne rentre ou sort de la file. Soit $(X_n)_{n \in \mathbb{N}}$ une suite à valeur dans $\mathbb{N}$. Ici, un pas de temps n représente un événement qui se passe. On va s'intéresser au fait que

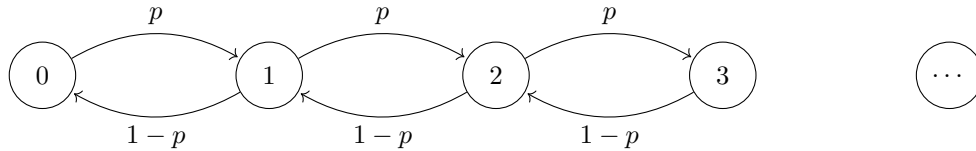


FIGURE 1.3 – File d'attente

la chaîne tende vers 0 ou $+\infty$ en fonction de $i_0 \in \llbracket 1, n \rrbracket$.

Exemple (Processus de Galton-Watson) Ici $(X_n)_{n \geq 1}$ compte le nombre d'individus d'une population dans la génération n . On suppose que chaque individu vivant a des descendants indépendamment des autres. On appelle $(\zeta_{n,k})_{n \leq 1, k \leq 1}$ la suite donnant le nombre de descendants de l'individu k vivant à la génération n .

On peut alors en déduire que $(\zeta_{n,k})_{n \leq 1, k \leq 1}$ est une suite de var iid et que :

$$X_{n+1} = \sum_{k=1}^{X_n} \zeta_{n,k}$$

1.2 Propriétés de Markov

Théorème (Version Faible) . Soit (X_n) une chaîne de Markov homogène de matrice de transition P et de loi initiale π_0 . Soit $n \in \mathbb{N}$ fixé. Conditionnellement à l'évènement $\{X_n = i\}$ pour tout $i \in E$, la suite de variables aléatoires $(X_{n+m})_{m \geq 0}$ est une chaîne de Markov homogène de même matrice de transition P et de loi initiale δ_i . De plus, elle est indépendante du passé de $(X_0, \dots, X_n)$ conditionnellement à $\{X_n = i\}$. Soient A un évènement indépendant de $X_{n+1}, \dots, X_{n+m}$ et B un évènement indépendant de $X_1, \dots, X_{n-1}$, on a alors :

$$\text{i.e. } \mathbb{P}(A \cap B | X_m = i) = \mathbb{P}(A | X_m = i) \mathbb{P}(B | X_m = i)$$

Démonstration Démontrons ce théorème en deux parties :

1. Égalité des lois : Montrons que :

$$\mathbb{P}(X_{m+1} = y_1, \dots, X_{m+n} = y_n | X_n = i, \dots, X_0 = i_0) = \mathbb{P}(X_1 = y_1, \dots, X_n = y_n | X_0 = i)$$

On a :

$$\begin{aligned} & \mathbb{P}(X_{m+1} = y_1, \dots, X_{m+n} = y_n | X_n = i, \dots, X_0 = i_0) \\ &= \frac{\mathbb{P}(X_{m+1} = y_1, \dots, X_{m+n} = y_n, X_n = i, \dots, X_0 = x_0)}{\mathbb{P}(X_m = i, X_{m-1} = x_{m-1}, \dots, X_0 = x_0)} \\ &= \frac{\pi_0(x_0) P(x_0, x_1) \dots P(X_{n-1}, i) P(i, y_1) \dots P(y_{n-1}, y_n)}{\pi_0(x_0) P(x_0, x_1) \dots P(x_{m-1}, i)} \\ &= P(i, y_1) P(y_1, y_2) \dots P(y_{n-1}, y_n) \\ &= \mathbb{P}(X_1 = y_1, \dots, X_n = y_n | X_0 = i) \end{aligned}$$

2. Indépendance conditionnelle : Soient A un évènement indépendant de $X_{n+1}, \dots, X_{n+m}$ et B un évènement indépendant de $X_1, \dots, X_{n-1}$. Montrons que :

$$\mathbb{P}(A \cap B | X_m = i) = \mathbb{P}(A | X_m = i) \mathbb{P}(B | X_m = i)$$

On a :

$$\mathbb{P}(A \cap B | X_m = i) = \frac{\mathbb{P}(A \cap B, X_m = i)}{\mathbb{P}(X_m = i)}$$

$$\begin{aligned}
& \mathbb{P}(A \cap B, X_m = i) \\
&= \sum_{\substack{x_0, \dots, x_{n-1} \in B \\ y_1, \dots, y_n \in A}} \mathbb{P}(X_0 = x_0, \dots, X_{m-1} = x_{m-1}, X_m = i, X_{m+1} = y_1, \dots, X_{m+n} = y_n) \\
&= \sum_{\substack{x_0, \dots, x_{n-1} \in B \\ y_1, \dots, y_n \in A}} \mathbb{P}(X_{n+1} = y_1, \dots, X_{n+m} = y_n | X_0 = i) \mathbb{P}(X_m = i, \dots, X_0 = x_0) \\
&= \left(\sum_{y_i \in A} \mathbb{P}(X_{n+1} = y_1, \dots, X_{n+m} = y_n | X_0 = i) \right) \left(\sum_{x_i \in B} \mathbb{P}(X_n = i, \dots, X_0 = x_0) \right) \\
&= \left(\sum_{y_i \in A} \mathbb{P}(A | X_n = i) \right) \left(\sum_{x_i \in B} \mathbb{P}(B, X_m = i) \right)
\end{aligned}$$

D'où :

$$\begin{aligned}
\mathbb{P}(A \cap B | X_m = i) &= \mathbb{P}(A | X_n = i) \mathbb{P}(B, X_n = i) \times \frac{1}{\mathbb{P}(X_m = i)} \\
&= \mathbb{P}(A | X_m = i) \mathbb{P}(B | X_m = i)
\end{aligned}$$

Remarque Cette propriété de Markov est aussi vraie de façon plus générale quand on conditionne par l'état de la chaîne en un temps aléatoire appelé temps d'arrêt. C'est ce que nous verrons par la suite.

Définition (Tribu $\mathcal{F}_n$) . On définit $\mathcal{F}_n$ la tribu engendrée par les variables aléatoires $(X_1, \dots, X_n)$ la tribu :

$$\begin{aligned}
\mathcal{F}_n &= \sigma(X_0, \dots, X_n) \\
&= \sigma(\{X_0 = x_0, \dots, X_n = x_n\}, x_0, \dots, x_n \in E)
\end{aligned}$$

Définition (Temps d'arrêt) . Un temps d'arrêt τ d'une chaîne de Markov $(X_n)_{n \geq 0}$ est une *variable aléatoire* à valeurs dans $\mathbb{N}$ mesurable par rapport à la tribu $\mathcal{F}_n$.

$$\text{i.e } \forall n \in \mathbb{N}, \{\tau \leq n\} \in \mathcal{F}_n$$

Cela signifie que $\{\tau \leq n\}$ ne dépend que des variables $X_0, \dots, X_n$.

Exemple Soit (X_n) une chaîne de Markov sur $\mathbb{Z}$. On définit le temps d'attente du dépassement de $j \in \mathbb{Z}$ comme la variable aléatoire :

$$\tau_j = \min\{i \geq 0 | X_i > j\}$$

τ_j est bien un temps d'arrêt pour la CMH car :

$$\{\tau_j \leq n\} = \{\exists k \in \mathbb{N}, X_k \geq j\}$$

donc τ_j ne dépend que de $X_0, \dots, X_n$. De façon générale, soit $A \subset E$, le temps d'attente de A défini par :

$$\tau_A = \min\{i \geq 0, X_i \in A\}$$

est un temps d'arrêt. Si cet ensemble est vide (i.e la chaîne de Markov n'atteint jamais A), alors $\tau_A = +\infty$. Par contre, $S_j = \max\{i \geq 0, X_i > j\}$ n'est pas un temps d'arrêt car $\{S_j \leq n\} = X_{n+1} < j, X_{n+2} < j, \dots$ dépend de ce qui s'est passé "après".

Théorème (Propriété de Markov forte) . Soit $(X_n)_{n \geq 0}$ une chaîne de Markov homogène et T un temps d'arrêt. Alors conditionnellement à $\{T < \infty\}$ et $\{X_T = x\}$ la chaîne décalée $(X_{T+n})_{n \geq 0}$ est encore une chaîne de Markov de condition initiale x . De plus, conditionnellement à $\{T < \infty\} \cap \{X_T = x\}$ on a :

$$(X_{T+n})_{n \geq 0} \text{ est indépendante de } (X_0, \dots, X_{T-1})$$

Simulation Aléatoire

Chapitre 1

Simulation de variables aléatoires

Contents

1.1 Simulation de variables aléatoires discrètes	14
--	----

La simulation peut se définir comme la prédiction de l'évolution d'un système en résolvant des équations venues de domaines appliqués. La notion d'aléatoire vient du fait que l'on simule des phénomènes dont nous n'aurons que des informations imparfaites tels que des phénomènes de désintégration nucléaire ou de prévention des risques. Les utilisations possibles de la simulation sont :

- Étude d'un phénomène (ex : la trajectoire d'un avion dans des turbulences)
- L'estimation de caractéristiques d'un système (ex : déterminer les caractéristiques d'une simulation météo)
- Tester des hypothèses (ex : en médecine, on peut utiliser la simulation aléatoire pour déterminer à l'avance les effets d'un médicament sur un patient.)
- La prise de décision dans un environnement incertain.
- Réaliser des expériences coûteuses voire impossibles (ex : effectuer des essais nucléaires informatiquement en fonction des essais passés ou simuler une propagation virale dans une population).

La simulation aléatoire est donc un outil rapide économique et contrôlé.

Définition (Simuler une variable aléatoire) . Simuler une variable aléatoire de loi μ consiste à produire un algorithme dont les réalisations donnent des copies indépendantes d'une variable de loi μ .

La simulation aléatoire est impossible sans l'accès à des *données aléatoires*. En pratique, on se contentera de données *pseudo-aléatoires* tel que le nombre de secondes écoulées depuis le 01/01/1070, la température d'un processeur ou les dernières touches entrées avant l'exécution du programme. Ceci est obtenu en **Python** avec la fonction `rand()`. Dans ce cours, on considère que chaque appel à cette fonction nous renvoie une variable iid de loi Uniforme sur $[0, 1]$.

Simuler une variable aléatoire consiste donc à construire un algorithme qui transforme un nombre fini de variables aléatoires iid de loi $\mathcal{U}([0, 1])$ en une variable aléatoire de loi μ :

$$\mathcal{U}([0, 1]) \xrightarrow{\text{simulation}} \mu$$

1.1 Simulation de variables aléatoires discrètes

Exemple (Simuler une variable aléatoire de loi $\mathcal{B}(p)$) Soit $X \sim \mathcal{U}([0, 1])$ avec $p \in [0, 1]$. Alors la variable $Y = \mathbb{1}_{X \leq p}$ suit une loi de Bernoulli de paramètre p .

$$\mathbb{P}(Y = 1) = \mathbb{P}(\mathbb{1}_{X \leq p} = 1) = \mathbb{P}(X \leq p) = p$$

$$\mathbb{P}(Y = 0) = 1 - p$$

On peut donc développer l'algorithme suivant :

```

1 def bernouilli(p):
2     x = rand()
3     if u <= p:
4         return 1
5     return 0
6

```

Exemple (Simuler une loi binomiale $\mathcal{B}(n, p)$) Soient $(X_1, \dots, X_n)$ une suite de variables aléatoires iid de loi $\mathcal{B}(n, p)$. La variable $Y := \sum_{i=1}^n X_i \sim \mathcal{B}(n, p)$ telle que :

$$\mathbb{P}(Y = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

Il nous faut donc seulement générer la somme de n variables aléatoires suivant une loi de Bernoulli. On a donc :

```

1 def binom(n, p):
2     s = 0
3     for i in range(n):
4         s += 1 * (rand() <= p)
5     return s
6

```

On cherchera à développer des programmes qui terminent en temps fini. On préférera deux qui terminent en temps borné (comme `binom(n, p)`).

Proposition Soit $(p_n)_{n \geq 1}$ une suite réelle positive telle que :

$$\sum_{k=1}^{\infty} p_k = 1$$

et (x_n) une suite d'éléments d'un ensemble E . On définit la loi μ par :

$$\mu = \sum_{k=1}^{\infty} p_k \delta_{x_k}$$

si une variable aléatoire X est de loi μ on aura alors $\mathbb{P}(X = x_k) = p_k$. On pose $f_0 = 0$ et $s_k = p_1 + \dots + p_k$. Soit U une variable aléatoire de loi $\mathcal{U}([0, 1])$. Alors la variable :

$$Y = \sum_{k=1}^{\infty} x_k \times \mathbb{1}_{s_{k-1} \leq U \leq s_k}$$

suit la loi μ . L'algorithme suivant simule une variable aléatoire de loi μ .

```

1 def simulation_mu([p1, ..., pk],[x1, ..., xk]):
2     u = rand()
3     s = p1
4     while u > s :
5         i++
6         s += pi
7     return xi
8

```

Démonstration Pour tout $n \in \mathbb{N}$, on a :

$$\mathbb{P}(s_{n-1} \leq U \leq s_n) = s_n - s_{n-1} = p_n$$

donc $\mathbb{P}(Y = x_n) = \mathbb{P}(s_{n-1} \leq U \leq s_n) = p_n$.

Exemple (Simuler une loi de Poisson $\mathcal{P}(\lambda)$) Pour rappel, si $X \sim \mathcal{P}(\lambda)$ alors :

$$\forall k \in \mathbb{N}, \quad \mathbb{P}(X = k) = e^{-\lambda} \frac{\lambda^k}{k!}$$

Posons $p_k := e^{-\lambda} \frac{\lambda^k}{k!}$. On remarque que :

$$p_{k+1} = e^{-\lambda} \frac{\lambda^{k+1}}{(k+1)!} = p_k \frac{\lambda}{k+1}$$

On en déduit donc l'algorithme suivant :

```

1 def poisson(lambda):
2     x = 0
3     p = exp(-\lambda)
4     s = p
5     u = rand()
6     while u > s :
7         x += 1
8         p = p * lambda / x
9         s += p
10    return x
11

```

Exemple (Loi uniforme) Simulons une loi uniforme sur $\{1, \dots, n\}$. On a donc l'algorithme suivant :

```

1 def uniforme(n):
2     x = 1
3     s = x/n
4     u = rand()
5     while u > s :
6         x += 1
7         s += 1/n
8     return x
9
10 def uniforme(n):
11     return ceil(n * rand(u))
12

```

Proposition (Algorithme du Rejet) Soit $(E, \mathcal{E}, \mu)$ un espace probabilisé et $A \subset E$ un événement tel que $\mu(A) > 0$. La loi μ conditionnellement à A , notée μ_A , est définie par :

$$\mu_A(B) = \frac{\mu(A \cap B)}{\mu(A)}, \quad B \in \mathcal{E}.$$

Soit $(X_n)_{n \geq 1}$ une suite de variables aléatoires i.i.d. de loi μ . On pose :

$$T := \inf\{n \geq 1 \mid X_n \in A\},$$

c'est-à-dire le rang du premier tirage appartenant à A . Alors :

1. $T \sim \mathcal{G}(\mu(A))$, c'est-à-dire

$$\mathbb{P}(T = k) = (1 - \mu(A))^{k-1} \mu(A), \quad k = 1, 2, \dots$$

2. $X_T \sim \mu_A$ (loi conditionnelle sur A)
3. X_T et T sont indépendantes.

Démonstration Soit $B \in \mathcal{E}$ et $k \geq 1$. On calcule :

$$\begin{aligned} \mathbb{P}(X_T \in B, T = k) &= \mathbb{P}(X_k \in B, T = k) \\ &= \mathbb{P}(X_1 \notin A, \dots, X_{k-1} \notin A, X_k \in A, X_k \in B) \\ &= \mathbb{P}(X_1 \notin A)^{k-1} \times \mathbb{P}(X_1 \in A \cap B) \\ &= (1 - \mu(A))^{k-1} \mu(A) \frac{\mu(A \cap B)}{\mu(A)} \\ &= \underbrace{(1 - \mu(A))^{k-1} \mu(A)}_{\mathbb{P}(T=k)} \underbrace{\frac{\mu(A \cap B)}{\mu(A)}}_{\mu_A(B)}. \end{aligned}$$

On en déduit :

- $\mathbb{P}(T = k) = (1 - \mu(A))^{k-1} \mu(A)$ (loi géométrique)
- $\mathbb{P}(X_T \in B) = \mu_A(B)$
- $\mathbb{P}(X_T \in B, T = k) = \mathbb{P}(T = k) \mathbb{P}(X_T \in B)$, donc T et X_T sont indépendantes.

Interprétation. L'algorithme du rejet décrit la procédure suivante :

1. Tirer $X_1, X_2, \dots$ de manière indépendante selon la loi μ .
2. Rejeter tout tirage $X_n \notin A$ et recommencer.
3. Accepter le premier $X_T \in A$: sa loi est exactement μ_A .

Ainsi :

- T représente le nombre de tirages nécessaires avant d'obtenir un succès.
- X_T est la valeur de ce premier succès, distribuée comme μ conditionnellement à A .
- T et X_T sont indépendants : le temps d'attente n'influence pas la valeur obtenue.

Exemple Si E est l'ensemble des notes d'étudiants, μ la distribution des notes et $A = \{\text{note} > 15\}$, alors :

- T = nombre d'étudiants observés avant d'en trouver un avec une note > 15 ,
- X_T = note de ce premier étudiant (loi conditionnelle sur A).

La loi de T est géométrique et la loi de X_T est la loi des notes sachant qu'elles sont > 15 .

Exemple On veut simuler une variable aléatoire U de loi uniforme sur les nombres premiers inférieurs à n . D'après la proposition précédente, on peut donc utiliser l'algorithme suivant :

```

1 def unif_prem(n):
2     x = uniforme(n)
3     while not premier(x):
4         x = uniforme(n)
5     return x
6

```

Ainsi le premier nombre premier sur lequel on tombe est uniformément distribué sur l'ensemble des nombres premiers inférieurs ou égaux à n .

Optimisation

Chapitre 1

Optimisation sans contraintes

Contents

1.1	CNS d'optimalité dans contraintes	18
1.1.1	Conditions Nécessaires	19
1.1.2	Conditions Suffisantes	19
1.2	Algorithmes de Descente	20
1.2.1	Principe	20
1.2.2	Plus profonde descente	21
1.2.3	Recherche Linéaire : Conditions	21
1.2.4	Algorithme de Recherche Linéaire	22
1.2.5	Méthode de Newton	22

En informatique et en mathématiques, l'optimisation est la recherche d'un optimum pour un problème posé. On s'intéresse ici à optimisation dite *continue*. On peut alors formuler un tel problème sous la forme suivante :

$$\begin{array}{ll} \text{minimize} & f(x) \\ \text{subject to} & x \in \mathcal{C} \end{array} \qquad \begin{array}{ll} \min & f(x) \\ \text{s.t.} & x \in \mathcal{C} \end{array}$$

On appelle x la *variable de décision* ou *variable de contrôle*. La fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est appelée *fonction objectif* ou le *critère*. L'ensemble $C \subset \mathbb{R}^n$ est appelée *ensemble des contraintes*.

Résoudre un tel problème signifie trouver $\bar{x} \in C \subset \mathbb{R}^n$ tel que :

$$\forall x \in C, \quad f(x) \geq \bar{x}$$

Remarque On peut remarquer que maximiser f revient à minimiser $-f$. On peut donc toujours se ramener à un problème de minisation. De plus, lorsque $C = \mathbb{R}^n$ on dit que le problème est *non contraint*.

1.1 CNS d'optimalité dans contraintes

Dans cette section, nous nous intéressons aux problèmes de la forme suivante :

$$\begin{array}{ll} \min & f(x) \\ \text{s.t.} & x \in \mathbb{R}^n \end{array}$$

où $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Nous commencerons par aborder l'optimalité locale puis nous passerons, grâce à la convexité, à l'optimalité globale.

1.1.1 Conditions Nécessaires

Évoquons tout d'abord quelques conditions nécessaires d'optimalité globale.

Théorème (CN1 - Principe de Fermat) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ et $\bar{x} \in \mathbb{R}^n$ un minimum local de f . Si f est différentiable alors :

$$\nabla f(\bar{x}) = 0$$

Définition (Matrice définie positive) . Soit $A \in \mathcal{M}_{n,n}(\mathbb{R})$ une matrice, on dit que A est *définie positive* si

$$\begin{aligned} \forall x \in \mathbb{R}^n, x \neq 0, \quad {}^t x A x > 0 \\ \iff Sp(A) \subset \mathbb{R}_+^* \end{aligned}$$

On note alors $A \succ 0$

Définition (Matrice semi-définie positive) . Soit $A \in \mathcal{M}_{n,n}(\mathbb{R})$ une matrice, on dit que A est *semi-définie positive* si

$$\begin{aligned} \forall x \in \mathbb{R}^n, x \neq 0, \quad {}^t x A x \geq 0 \\ \iff Sp(A) \subset \mathbb{R}_+ \end{aligned}$$

On note alors $A \succeq 0$

Théorème (CN2) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une fonction quelconque et soit $\bar{x} \in \mathbb{R}^n$ un minimum local de f . Si f est deux fois différentiable en $\bar{x}$ alors :

$$\nabla^2 f(\bar{x}) \succeq 0$$

(i.e ssi la hessienne de f est semi-définie positive). On appelle cette condition la *condition d'optimalité du second ordre*.

1.1.2 Conditions Suffisantes

Passons maintenant aux conditions suffisantes d'optimalités...

Théorème (CS1) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une fonction quelconque et $\bar{x} \in \mathbb{R}^n$. On suppose qu'il existe $r > 0$ tel que :

- f est différentiable en tout point de $B(\bar{x}, r)$
- $\forall x \in B(\bar{x}, r) \setminus \{\bar{x}\}, \langle \nabla f(x), x - \bar{x} \rangle \geq 0$

Alors $\bar{x}$ est un *minimum local* de f .

Théorème (CS2) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une fonction quelconque et $\bar{x} \in \mathbb{R}^n$. Alors si :

- f est deux fois différentiable en $\bar{x}$
- $\nabla f(\bar{x}) = 0$
- $\nabla^2 f(\bar{x}) \succeq 0$

Alors $\bar{x}$ est un *minimum local strict* de f .

Le théorème suivant nous permet de généraliser ce minimum local en minimum global. Pour cela, il faut que la fonction ne décroît plus (i.e que $\bar{x}$ soit en bas de la parabole). Pour cela, on utilisera la notion de convexité.

Théorème (Convexité et Minimum) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une fonction quelconque et $\bar{x} \in \mathbb{R}^n$ un minimum local de f . Alors :

1. Si f est *convexe* alors $\bar{x}$ est un minimum *global* de f .
2. Si f est *strictement convexe* alors $\bar{x}$ est l'*unique minimum global* de f .

1.2 Algorithmes de Descente

1.2.1 Principe

Nous allons ici nous intéresser à des algorithmes permettant d'approximer un minimum d'un champ de scalaires. Ces algorithmes seront itératifs, autrement dit, à chaque pas de temps, nous approcherons de plus en plus notre solution. On les appelle des *algorithmes de descente*. La notion de "descente" se formalise par la construction d'une suite $(u_n)_{n \in \mathbb{N}}$ tel que :

$$\forall n \in \mathbb{N}, \quad f(u_{n+1}) \leq f(u_n)$$

Définition (Direction de descente) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une application différentiable et $x, d \in \mathbb{R}^n$. Le vecteur d est la *direction de descente* de f en x si :

$$\langle \nabla f(x), d \rangle < 0$$

Pour l'algorithme de descente, ce vecteur d nous donnera le "sens" dans lequel il faut descendre pour s'approcher du minimum. Or maintenant, que l'on sait de quel "coté" descendre, essayons de savoir "de combien" faut-il descendre.

Théorème (Existence du pas) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une application différentiable. Soient $x, d \in \mathbb{R}^n$ tels que $\nabla f(x) \neq 0$. Si d est un direction de descente de f en x , alors il existe $\mu > 0$ tel que :

$$\forall \alpha \in]0, \mu[, \quad f(x + \alpha d) < f(x)$$

De plus, pour tout $\beta < 1$, il existe $\mu > 0$ tel que :

$$\forall \alpha \in]0, \mu[, \quad f(x + \alpha d) < f(x) + \alpha \beta \langle \nabla f(x), d \rangle$$

On appellera α le *pas* de l'itération.

Proposition Chaque itération du processus de descente se fait donc en 3 étapes :

1. Trouver une direction d_k telle que $\langle \nabla f(x_k), d_k \rangle < 0$
2. Trouver un pas α_k tel que $f(x_k + \alpha_k d_k) < f(x_k)$
3. Calculer $x_{k+1} = x_k + \alpha_k d_k$

1.2.2 Plus profonde descente

Nous allons maintenant nous intéresser à différents types de descente. En effet, en fonction de l'application à minimiser nous utiliserons différents pas et directions lors de la descente. La méthode naturelle serait de prendre la direction dans laquelle les valeurs de l'application descendent le plus. Nous choisirons donc au début la direction $d = -\nabla f(x)$.

Théorème (Gradient comme direction) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une application différentiable. Soient $x \in \mathbb{R}^n$ alors pour tout $d \in \mathbb{R}^n$ tel que $\|d\| = \|\nabla f(x)\|$ on a :

$$\langle \nabla f(x), -\nabla f(x) \rangle \leq \langle \nabla f(x), d \rangle$$

On dira alors que $d^* = -\nabla f(x)$ est la *direction de plus profonde descente*.

Proposition Pour mettre en oeuvre notre plus profonde descente, à chaque itération, nous calculons donc :

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

Un théorème précédent nous assure de pouvoir trouver un α_k qui convient à chaque itération. Pour une valeur de α_k optimale, on pourrait résoudre le problème suivant à chaque itération :

$$\begin{aligned} \min \quad & f(x_k + \alpha d_k) \\ \text{s.t.} \quad & \alpha > 0 \end{aligned}$$

Mais nous ne sommes pas sûr de toujours pouvoir le résoudre.

1.2.3 Recherche Linéaire : Conditions

On remarque assez vite que choisir un pas constant très petit (comme 0.0001) rallonge considérablement la descente. D'autre part, nous savons que l'existence d'un minimum pour un champ de scalaire est une notion locale. Prendre un pas trop grand risquerai de nous faire rater le minimum et de faire augmenter la fonction objectif.

Or, résoudre un problème d'optimisation pour α_k à chaque itération n'est pas non plus une bonne solution car elle augmente la complexité de la méthode. Nous allons donc essayer de développer une méthode qui permet de résoudre approximativement la recherche d'un α_k à moindre coût. Pour cela, nous choisirons α_k en fonction de la rapidité ou non de la décroissance de notre fonction objectif.

Proposition Pour trouver un pas raisonnable à chaque itération, nous utiliserons l'information contenue dans le gradient de la fonction objectif. En effet, celui-ci nous permet de savoir comment celle-ci diminue et à quelle vitesse. On regardera, pour chaque pas possible, la *diminution* qu'il produit dans notre algorithme :

$$f(x_k) - f(x_k + \alpha_k d_k) > \gamma \alpha_k, \quad \text{pour } \gamma > 0$$

Diminution qui sera proportionnelle au pas. Ainsi, plus la pente $\langle \nabla f(x_k), d \rangle$ de la fonction objectif dans la direction de d sera importante, plus le pas sera grand.

Définition (Première condition de Wolfe - Condition d'Arjimo) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une application différentiable. Soient $x_k \in \mathbb{R}^n$ et $d_k \in \mathbb{R}^n$ une direction de descente de f en x_k et $\alpha_k > 0$. On dit que la fonction f *diminue suffisamment* en $x_k + \alpha_k d_k$ par rapport à x_k si :

$$f(x_k + \alpha_k d_k) < f(x_k) + \alpha_k \beta \langle \nabla f(x_k), d_k \rangle$$

Il peut arriver que les pas α_k deviennent très proches de 0 et empêchent la méthode de progresser. On va donc chercher à avancer dans notre descente mais sans dépasser le minimum (première condition de Wolfe), mais en avançant suffisamment pour arriver sur un point où la pente de la fonction objectif est moins raide (seconde condition de Wolfe).

Définition (Seconde Condition de Wolfe) . Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une application différentiable. Soit $x_k \in \mathbb{R}^n$, $d_k \in \mathbb{R}^n$ une direction de descente de f en x_k et $\alpha_k > 0$. On dira que le point $x_k + \alpha_k d_k$ apporte une progression suffisante par rapport à x_k si :

$$\begin{aligned} \langle \nabla f(x_k + \alpha_k d_k), d_k \rangle &\geq \beta \langle \nabla f(x_k), d_k \rangle \\ \iff \frac{\langle \nabla f(x_k + \alpha_k d_k), d_k \rangle}{\langle \nabla f(x_k), d_k \rangle} &\geq \beta \end{aligned}$$

La recherche linéaire permet donc d'identifier un pas qui satisfait les deux conditions de Wolfe.

1.2.4 Algorithme de Recherche Linéaire

L'idée de l'algorithme de recherche linéaire est de construire un intervalle $[\alpha_g, \alpha_d]$ contenant des valeurs qui satisfassent les deux conditions de Wolfe. On choisit ensuite un $\alpha \in [\alpha_g, \alpha_d]$ et s'il viole l'une des deux conditions de Wolfe, on réduit l'intervalle précédente et on recommence.

Proposition Soient $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une application différentiable, $x_k \in \mathbb{R}^n$, une direction de descente $d_k \in \mathbb{R}^n$ de f par rapport à x_k et $\alpha_0 > 0$. Soient β_1 et β_2 deux paramètres tels que $0 < \beta_1 < \beta_2 < 1$ et $\lambda > 1$. On a l'algorithme suivant :

1. **Initialisation** : $i = 0, \alpha_g = 0, \alpha_d = \infty$
2. **Itération** : Tant que l'un des conditions de Wolfe est violée avec $\alpha_k = \alpha_i$ faire :
 - (a) si α_i viole la première condition de Wolfe, alors le pas est trop long donc :

$$\begin{aligned} \alpha_d &= \alpha_i \\ \alpha_{i+1} &= \frac{\alpha_g + \alpha_d}{2} \end{aligned}$$

- (b) si α_i viole la seconde condition de Wolfe, alors le pas est trop court donc :

$$\begin{aligned} \alpha_g &= \alpha_i \\ \alpha_{i+1} &= \begin{cases} \frac{\alpha_g + \alpha_d}{2} & \text{si } \alpha_d < \infty \\ \lambda \alpha_i & \text{sinon} \end{cases} \end{aligned}$$

- (c) $i++$

Théorème (Finitude de la recherche linéaire) . Soit f une application différentiable bornée inférieurement dans une direction d_k par rapport à $x_k \in \mathbb{R}^n$. Alors la recherche linéaire d'un pas se termine après un nombre fini d'itérations.

1.2.5 Méthode de Newton

Algorithmique

Chapitre 1

Flot maximal dans un graphe

Contents

1.1	Rappels sur les graphes	24
1.2	Cycle et flot	26
1.3	Flot et coupure	27
1.3.1	Flot maximal dans un réseau	27
1.3.2	Théorème de Coupure minimale	28
1.4	Flot maximal : principe de Ford-Fulkerson	29

L'objectif de ce chapitre est d'être capable d'organiser efficacement un flot d'objets dans un réseau. Par exemple, on souhaite amener le maximum de camions d'un point A à un point B en passant par certaines routes. De même, on souhaiterait savoir comment router efficacement le plus de paquets dans un réseau informatique.

Tout ces problèmes se généralisent en ce que l'on appelle la *recherche de flot maximal dans un graphe orienté*. Nous allons ici définir proprement ce genre de problème, donner des conditions d'existence de flot maximal et proposer des algorithmes pour les construire.

1.1 Rappels sur les graphes

Pour représenter une structure de réseau (routier, informatique, etc...), on utilise les graphes. Ce cours fait donc suite à celui sur la théorie des graphes enseigné en licence. Effectuons quelques rappels.

Définition (Graphe Orienté) . On appelle graphe orienté un couple d'ensembles finis $G = (X, E)$ où $X = \{1, \dots, n\}$ représente les sommets du graphe et $E = \{(x_i, y_j), \dots\}$, $x_i, y_j \in X$ l'ensemble ordonné des arrêtes du graphe. Pour un graphe orienté, les notions de graphe simple et multigraphes sont les même que pour le cas non orienté.

Exemple (Graphes et leur représentation gravarphique) Soient G_1 et G_2 deux graphes, en voici une représentation :

Définition (Voisinage) . Le voisinage d'un sommet x de G est l'ensemble des sommets y de G tels qu'il existe une arrête entre x et y dans G . On le note $V(x)$.

Remarque Si x est dans le voisinage de y , on dira que x est adjacent à y et inversement.

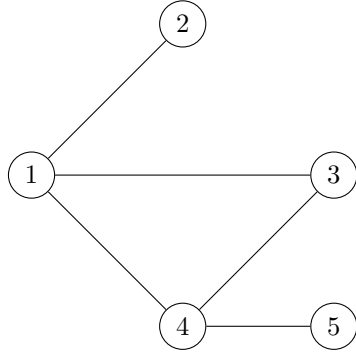


FIGURE 1.1 – Graphe non orienté

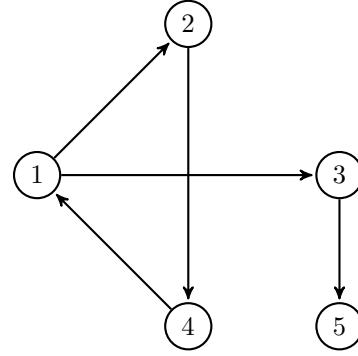


FIGURE 1.2 – Graphe orienté

Définition (Degré) . Le degré d'un sommet x de G est le cardinal du voisinage x . On le note $d(x)$. Un sommet de voisinage nul est dit *isolé* et un sommet de voisinage égal à 1 est dit *pendant*.

Définition (Voisinage entrant et sortant) . Soit G un graphe orienté et x un sommet de G . On définit deux types de voisinages :

- *Voisinage entrant* : noté $V^-(x)$ est l'ensemble des prédécesseurs de x .
- *Voisinage sortant* : noté $V^+(x)$ est l'ensemble des successeurs de x .

On définira comme précédemment le degré sortant et le degré entrant d'un sommet x .

Définition (Chaîne) . On appelle chaîne de $G = (X, E)$ de longueur n toute suite alternée de sommets et d'arrêtes de G telle que :

$$c = (x_0, a_1, x_1, \dots, a_n, x_n), \text{ telle que } \forall i \in \llbracket 1, n \rrbracket, a_i = (x_{i-1}, x_i)$$

Ici, n représente le nombre d'arrêtes de la chaîne. Dans le cas d'un graphe simple, on notera les chaînes de la façon suivante :

$$c = (x_0, \dots, x_n) \quad \text{ou} \quad c = x_0 - x_1 - \dots - x_n$$

Dans le cas des graphes orientés, on notera une chaîne simplement avec les arrêtes dont elle est constituée :

$$c = (u_1, \dots, u_n), \forall i \in \llbracket 1, n \rrbracket u_i \in E$$

Définition (Accessibilité) . Soit $G = (X, E)$ un graphe. On a :

- Soient x et y deux sommets de G . On dit que y est accessible à partir de x s'il existe une chaîne joignant x et y dans G .
- G est dit *connexe* ssi $\forall x \in X, \forall y \in X, y$ est accessible à partir de x .
- L'accessibilité est une relation d'équivalence entre les sommets. Ses classes d'équivalences sont les composantes connexes de G .

Définition (Chaîne Simple) . Une chaîne est *simple* si elle ne passe pas deux fois par la même *arrête* et elle est dite *élémentaire* si elle ne passe pas deux fois par le même *sommet*. On remarquera facilement qu'une chaîne élémentaire est simple.

Définition (Cycle) . Un cycle de G est une chaîne simple dont le départ et l'arrivée sont le même sommet. Un cycle est donc de la forme :

$$c = x_0 - x_1 - \cdots - x_{n-1} - x_0$$

Théorème (Propriétés des cycles et chaînes) . Soit G un graphe à $n \in \mathbb{N}$ sommets.

- Toute chaîne élémentaire a une longueur inférieure à $n - 1$
- Toute cycle élémentaire a une longueur inférieure à n
- De toute chaîne, on peut en extraire une chaîne élémentaire.

1.2 Cycle et flot

Définissons maintenant la notion de flot dans un cas général.

Définition (Vecteur de chaîne) . Soit $c = (u_1, \dots, u_n)$ une *chaîne élémentaire* dans un graphe orienté $G = (X, E)$ tel que $|E| = n$ et d'arcs u_i . On appelle *vecteur de chaîne* associé à c l'application φ associée à c telle que :

$$\varphi : \begin{cases} E \longrightarrow \mathbb{Z} \\ u_i \longmapsto \begin{cases} 1 \text{ si } u_i \text{ est dans le sens de } c \\ -1 \text{ si } u_i \text{ n'est pas dans le sens de } c \\ 0 \text{ si } u_i \text{ n'est pas dans } c \end{cases} \end{cases}$$

On la note en général $\varphi = (\varphi(u_1), \varphi(u_2), \dots, \varphi(u_n)) \in \mathbb{Z}^n$.

Venons en maintenant à la définition phare de ce chapitre :

Définition (Flot) . Soit $G = (X, E, c)$ un graphe orienté. On appelle *flot* dans G tout vecteur de cycle φ de G qui vérifie la loi de Kirchoff :

$$\forall x \in X, \quad \sum_{u_i \in \omega^-(\{x\})} \varphi(u_i) = \sum_{u_i \in \omega^+(\{x\})} \varphi(u_i)$$

Et on note $\varphi^i = \varphi(u_i)$ le *flot* de l'arrête u_i .

Remarque Un flot est donc un chaîne élémentaire d'un graphe orienté dont, pour chaque noeud, la somme des flots sortants est égale à la somme des flots entrants.

Propriété (Flots) . Soit $G = (X, E, c)$ un graphe tel que $|E| = n$, alors :

- Le vecteur null de $\mathbb{Z}^n$ est un flot de G .
- Toute combinaison linéaire de flots de G est aussi un flot de G .
- Tout vecteur cyclique de G est un flot de G .

Exemple Soit G le graphe suivant : Le vecteur $\varphi_1 = (2, -2, 1, 3, 4, -1, 1)$ est un flot de G . Le

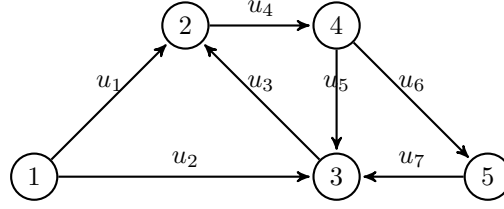


FIGURE 1.3 – flot dans un graphe

vecteur $\varphi_2 = (0, 0, 0, 0, 1, -1, 1)$ est aussi un flot. Soit $v = 2\varphi_1 + 3\varphi_2$, c'est aussi un flot de G .

Proposition (Loi de Kirchoff généralisée) Soit $G = (X, E, c)$ et $A \subset E$ non vide. Un chaîne φ est un flot de G ssi :

$$\sum_{u \in \omega^-(A)} \varphi(u) = \sum_{u \in \omega^+(A)} \varphi(u)$$

1.3 Flot et coupure

1.3.1 Flot maximal dans un réseau

Dans cette section, nous revenons à notre problème de départ en définissant la notion de graphe de transport. Graphes dans lesquels nous chercherons à construire des flots maximaux.

Définition (Réseau de Transport) . Un *réseau de transport* est un graphe orienté pondéré $R = (X, E, c)$ qui contient :

- un sommet sans prédécesseur appelé *source* (noté s).
- un sommet sans fils appelé *puits* (noté t).
- c , une application, qui à chaque arc, lui associe sa capacité notée c :

$$c : \begin{cases} E \longrightarrow \mathbb{R} \cup \{\infty\} \\ u = (x, y) \longmapsto c(u) \end{cases}$$

- un arc fictif de t à s doté d'une capacité infinie, appelé *arc retour*.

Définition (Flot compatible) . Un *flot compatible* sur un réseau $R = (X, E, c)$ tel que $|E| = n$ est un vecteur $\varphi \in \mathbb{Z}^n$ qui vérifie :

- **(Conservation de Kirchhoff)** pour tout $x \in X \setminus \{s, t\}$ (hors source s et puits t), on a :

$$\sum_{u \in \omega^+(x)} \varphi(u) = \sum_{u \in \omega^-(x)} \varphi(u)$$

où $\omega^+(x)$ (resp. $\omega^-(x)$) est l'ensemble des arcs sortants (resp. entrants) de x .

- **(Compatibilité avec les capacités)** :

$$\forall u \in E, \quad 0 \leq \varphi(u) \leq c(u).$$

La *valeur* du flot, notée $v(\varphi)$, est définie par :

$$v(\varphi) := \sum_{u \in \omega^+(s)} \varphi(u) - \sum_{u \in \omega^-(s)} \varphi(u),$$

c'est-à-dire le flux net sortant de la source (équivalent au flux net entrant dans le puits). Enfin, on dit qu'un arc u est *saturé* si $\varphi(u) = c(u)$.

Exemple Voici un exemple de réseau de transport :

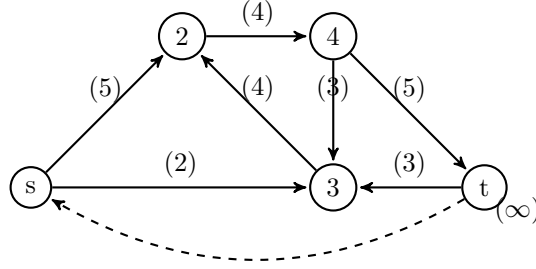


FIGURE 1.4 – Réseau de Transport

Définition (Flot maximal) . Un *flot maximal* d'un réseau est un flot compatible qui maximise la valeur $v(\varphi)$ parmi tous les flots compatibles du réseau.

1.3.2 Théorème de Coupure minimale

Pour construire un flot maximal dans un réseau, nous avons besoin d'un peu de théorie.

Définition (Coupure et cocycle) . Soit $R = (X, E, c)$ un réseau, où X est l'ensemble des sommets, E l'ensemble des arêtes et $c : E \rightarrow \mathbb{R}^+$ la fonction de capacité. Une *coupure* ct séparant s de t est un couple $(A, X \setminus A)$ tel que :

$$A \subset E, \quad s \in A, \quad t \notin A.$$

L'ensemble des arcs de la coupure, noté $\text{arcs}(ct)$ est l'ensemble :

$$\text{arcs}(ct) = \omega^+(A) := \{u = (i, j) \in E \mid i \in A, j \in X \setminus A\}.$$

Qui correspond à l'ensemble des arcs qui, si on les enlève, coupe tout les chemins entre s et t .

Définition (Capacité d'une coupure) . La *capacité* d'une coupure ct est la somme des capacités de ses arêtes :

$$\text{capa}(ct) := \sum_{u \in \text{arcs}(ct)} c(u).$$

Théorème (Flot et coupure minimale) . Pour tout flot compatible φ et pour toute coupure ct d'un réseau R , on a :

$$v(\varphi) \leq \text{capa}(ct)$$

1.4 Flot maximal : principe de Ford-Fulkerson

L'objectif de l'algorithme de Ford-Fulkerson est de construire de manière itérative un flot maximal. Pour cela, on part du flot nul dans le réseau. Et petit-à-petit, on cherche des chemins non saturés de la source s vers le puit t pour lesquels on peut augmenter leur flot. Une fois que l'on n'est plus capable de trouver de chemin, alors l'algorithme se termine. La définition ci-dessous détaille le processus de construction d'un tel chemin.

Définition (Principe de Ford-Fulkerson) . Soit $R = (X, E, c)$ un réseau et φ un flow sur R . L'algorithme de construction d'un chemin non saturé est :

1. **Initialisation** : On marque s
2. **Itération** : Pour tout sommet marqué $x, y \in X$ peut être marqué si :
 - y n'est pas encore marqué
 - $\begin{cases} \exists u = (x, y) \in E, \varphi(u) < c(u) & \text{(marquage direct)} \\ \exists u = (y, x) \in E, \varphi(u) < c(u) & \text{(marquage indirect)} \end{cases}$

L'algorithme se termine si on arrive à marquer le puit t . Dans ce cas, la chaîne produite est alors appelée *chaîne augmentante*, notée ch . On note ch^+ les sommets de ch marqués directement et ch^- les sommets marqués indirectement.

Propriété (Calcul d'un nouveau flot) . Soit $R = (X, E, c)$ un réseau et φ un flow sur R . Soit ch une chaîne augmentante de R . Il est alors possible d'augmenter la valeur du flot φ de $k \in \mathbb{R}$ en ajoutant à chaque arcs le flot défini par :

$$k = \min \left(\min_{u \in ch^+} \{c(u) - \varphi(u)\}, \min_{u \in ch^-} \{\varphi(u)\} \right)$$

Le nouveau flot est obtenu en augmentant de k le flow des arcs de ch^+ et en diminuant le flot de k des arcs de ch^- .

Théorème (Coupure Minimale - Flot Maximal) . Soit $R = (X, E, c)$ un réseau. Soit F l'ensemble des flots possibles sur R et K l'ensemble des coupures possibles de R . Alors la valeur du flot maximal est égale à la capacité de la coupure minimale.

$$\text{i.e } \max_{\varphi \in F} \{v(\varphi)\} = \min_{ct \in K} \{\text{capa}(ct)\}$$

Ce théorème nous permet donc de trouver directement la valeur du flot maximal que l'on peut faire passer dans notre réseau sans avoir à en construire un.

Compilation

Chapitre 1

Analyse Syntaxique

1.1 Analyse Syntaxique et approche descendante (Grammaires $LL(k)$)

Lors de la dérivation d'un mot par une grammaire, on peut se heurter à un problème de taille : quel règle de dérivation choisir pour obtenir un arbre de dérivation acceptant ? Nous allons donc introduire un type de grammaire nous permettant de régler ce problème. Un seul type de dérivation sera possible et nous saurons exactement combien de caractères regarder dans notre mot pour choisir la règle de dérivation à appliquer.

Les grammaires $LL(k)$ vont nous permettre de faire directement le lien avec le processus de *parsing*, indispensable en compilation.

1.1.1 Définition

Définition (Grammaire $LL(k)$) . Soit $k \in \mathbb{N}$ et G une grammaire. On dit que G est $LL(k)$ si on peut analyser n'importe quel mot de son langage de gauche à droite, en construisant la dérivation la plus à gauche, et en regardant au plus k symboles d'entrée à la fois, pour décider de manière univoque quelle production appliquer à chaque étape.

Remarque Détaillons un peu cette définition. La grammaire G est $LL(k)$ si elle satisfait trois critères principaux :

- **L (Left to right)** : Lorsque l'on dérive un mot par la grammaire, on veut pouvoir le lire de gauche à droite en commençant par le premier symbole.
- **L (Leftmost derivation)** : Lorsque l'on remplace une variable non terminale de l'expression en cours de dérivation, on veut que ce soit celle la plus à gauche.
- **k (k lookahead)** : k est le nombre de symbole que le parser peut regarder pour déterminer quelle règle utiliser sans les consommer.

Intuitivement, on cherche une grammaire prédictive. Pour chaque variable à chaque étape de la dérivation, le parser doit savoir exactement quelle règle appliquer en regardant k lettres. Évidemment, on ne considèrera pas de grammaire ambiguë. Nous allons donc développer un algorithme pour construire une *table de dérivation* qui nous donnera la règle à appliquer à chaque étape de l'analyse syntaxique.

1.1.2 Ensemble First, Follow, k -lookahead

Définition (k - Lookahead) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$ et une règle de production $A \rightarrow \alpha$. On définit l'ensemble $LA(k, A \rightarrow \alpha)$ comme l'ensemble des chaînes de longueur $\leq k$ qui déclenchent la règle de production $A \rightarrow \alpha$ tel que :

$$LA(k, A \rightarrow \alpha) := \{w \in \Sigma^{\leq k} \mid w \text{ peut apparaître si on choisit la règle } A \rightarrow \alpha\}$$

$LA(k, A \rightarrow \alpha)$ est donc l'ensemble des lettres que doit regarder le parser pour déterminer quelle règle de production appliquer. Son calcul permet aussi de vérifier qu'une grammaire est $LL(k)$. En effet si les k -lookahead des règles d'un même non terminal ont des mots en communs, la grammaire n'est pas $LL(k)$ (i.e on ne saura pas choisir quelle règle appliquer).

Définition (Ensemble First) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$. Soit $X \in V$ un état *non terminal*. On définit l'ensemble $First(X)$ comme l'ensemble des *éléments terminaux* de Σ qui peuvent apparaître en premier lors d'une dérivation de X . De plus, si X peut produire un ε , alors $\varepsilon \in First(X)$. Plus formellement :

$$First_k(\alpha) := \left\{ u \in \Sigma^*, \begin{cases} \text{si } |u| < k \text{ et } \alpha \rightarrow^* u \\ \text{si } |u| = k \text{ et } \alpha \rightarrow^* u\beta \text{ avec } \beta \in (V \cup \Sigma)^* \end{cases} \right\}$$

Cet ensemble permet de déterminer l'ensemble des mots de longueur inférieure ou égale à k qui peuvent dériver du mot auquel on l'applique. En général on commence le calcul à partir d'un non-terminal.

Propriété (Règles de la calcul (First)) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$, on a les règles de calcul suivantes :

- Soit $a \in \Sigma$ un terminal, on a : $First(a) = \{a\}$
- $First(\varepsilon) = \{\varepsilon\}$
- Soit $A \in V$ une variable avec les productions : $A \rightarrow \alpha_1 \mid \dots \mid \alpha_n$ on a :

$$First(A) = \bigcup_{i \in [1, n]} First(\alpha_i)$$

Exemple (Ensemble First) Soit G la grammaire aux règles de production suivantes :

$$\begin{cases} S' \rightarrow S\$ \\ S \rightarrow aAe \mid bc \mid \varepsilon \\ A \rightarrow d \mid \varepsilon \end{cases}$$

On a :

$$First_3(S) = First_3(aAe) \cup First_3(bc) \cap First_3(\varepsilon)$$

Or :

$$First_3(aAe) = a \cdot First_2(A) \cdot e = a\{d, \varepsilon\}e = \{ade, ae\}$$

D'où :

$$First_3(S) = \{ade, ae, bd, \varepsilon\}$$

Définition (Ensemble Follow) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$. Soit $X \in V$ un état *non terminal* et $k \in \mathbb{N}$. On définit $Follow_k(A)$ comme l'ensemble des mots de longueur exactement k qui peuvent suivre le non terminal A .

Propriété (Règle de calcul (Follow)) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$, on a alors :

$$Follow_k(A) = \bigcup_{Y \rightarrow \gamma A \delta} First_k(\delta Follow_k(Y))$$

Si $\varepsilon \in First_k(\delta)$, on ajoute aussi $Follow_k(Y)$.

Exemple (Ensemble Follow) Soit G la grammaire aux règles de production suivantes :

$$\begin{cases} S' \rightarrow S\$ \\ S \rightarrow aA|bABc \\ A \rightarrow d \\ B \rightarrow e|\varepsilon \end{cases}$$

On remarque que :

- $First_3(Bc) = First_3(B) \cdot c = \{ec, c\}$
- $Follow_3(S) = First_3(Follow_3(\$^3)) = \{\$^3\}$

$$\begin{aligned} Follow_3(A) &= First_3(Follow_3(S)) \cup First_3(BcFollow_3(S)) \\ &= \{\$^3\} \cup First_3(Bc \cdot Follow_3(S)) \\ &= \{\$^3\} \cup \{ec \cdot First_1(Follow_1(S))\} \cup \{c \cdot First_2(Follow_2(S))\} \\ &= \{\$^3\} \cup \{ec\$ \} \cup \{c\$^2\} \\ &= \{\$^3, ec\$, c\$^2\} \end{aligned}$$

Exemple (Follow avec récursivité) Soit G la grammaire aux règles de production suivantes :

$$\begin{cases} S' \rightarrow S\$^3 \\ S \rightarrow aA \\ A \rightarrow bAc|\varepsilon \end{cases}$$

On a ici :

$$\begin{aligned} Follow_3(A) &= First_3(Follow_3(S)) \cup First_3(Follow_3(A)) \\ &= First_3(\$^3 \cdot Follow_3(S')) \cup c \cdot First_2(Follow_2(A)) \\ &= \{\$^3\} \cup \{\$^2\} \cup c \cdot First_1(Follow_1(A)) \\ &= \{\$^3, c\$^2, cc\$, ccc\} \end{aligned}$$

1.1.3 Règles de calcul et table d'analyse

Théorème (Calcul de $LA(k)$) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$ et une règle de production $A \rightarrow \alpha$. On a :

$$LA(k, A \rightarrow \alpha) = First_k(\alpha) \cup \{x \in Follow_k(A) | \alpha \text{ peut dériver } \varepsilon\}$$

plus précisément :

- Si α ne peut pas dériver ε : $LA(k, A \rightarrow \alpha) = First_a(\alpha)$
- Si α peut dériver ε : $LA(k, A \rightarrow \alpha) = First_a(\alpha) \cup Follow_k(A)$

Proposition On peut réduire le théorème suivant à cette simple formule :

$$LA(k, A \rightarrow \alpha) = First_k(\alpha Follow_k(A))$$

L'ensemble de ces règles de calcul vont nous permettre de définir quelle règle de dérivation appliquer à chaque étape de l'analyse syntaxique d'un mot. Nous regrouperons toutes ces règles dans une table appelée la *table d'analyse*.

Définition (Table d'analyse) . Soit $G = (\Sigma, V, S, P)$ une grammaire $LL(k)$. Pour chaque règle de production $A \rightarrow \alpha$ et chaque $w \in LA(k, A \rightarrow \alpha)$, on définit :

$$M[A, w] = \alpha$$

Autrement dit, $M[A, w]$ contient la partie droite de la règle de production à utiliser quand le non-terminal courant est A et que les k prochains symboles de l'entrée forment w .

La table d'analyse d'une grammaire $LL(k)$ permet, pour chaque non-terminal A et chaque séquence de k symboles de lookahead w , de déterminer quelle production appliquer. Pour construire la table d'analyse d'une grammaire $LL(k)$, nous devons calculer les k -lookahead pour chaque règle de la grammaire.

Exemple Soit G la grammaire suivante :

$$\begin{cases} S' \rightarrow S\$^3 \\ S \rightarrow aAb \\ A \rightarrow c|\varepsilon \end{cases}$$

Calculons sa table d'analyse. Pour cela, commençons par le calcul des 3-lookahead :

$$\begin{aligned} LA(3, S' \rightarrow S\$^3) &= First_3(S\$^2 \cdots Follow_3(S)) \\ &= First_3(S\$^3) \\ &= First_3(acb\$^3) \cup First_3(ab\$^3) \\ &= \{acb, ab\$ \} \end{aligned}$$

$$\begin{aligned} LA(3, S \rightarrow aAb) &= First_3(aAb \cdot Follow_3(S)) \\ &= First_3(aAb \cdots First_3(\$^3)) \\ &= First_3(aAb\$^3) \\ &= First_3(acb\$^3) \cup First_3(ab\$^3) \\ &= \{acb, ab\$ \} \end{aligned}$$

$$\begin{aligned} LA(3, A \rightarrow c) &= First_3(c \cdot Follow_3(A)) \\ &= First_3(c \cdot First_3(b \cdot Follow_3(S))) \\ &= First_3(c \cdot First_3(c\$^3)) \\ &= First_3(c \cdot \$^2) = \{cb\$ \} \end{aligned}$$

$$\begin{aligned} LA(3, A \rightarrow \varepsilon) &= First_3(Follow_3(A)) \\ &= First_3(b\$^2) = \{b\$^2 \} \end{aligned}$$

On peut donc construire la table d'analyse de la grammaire :

Non-terminal	acb	$ab\$$	$cb\$$	$b\2
S'	$S' \rightarrow S\3	$S' \rightarrow S\3		
S	$S \rightarrow aAb$	$S \rightarrow aAb$		
A			$A \rightarrow c$	$A \rightarrow \epsilon$

Chaque cellule indique la production à appliquer si le flux d'entrée commence par la chaîne correspondante de longueur 3.

Ces nouveaux modèles de grammaires, quoique puissants ne permettent pas d'effectuer l'analyse syntaxique de n'importe quel langage. En effet, toutes les grammaires ne sont pas $LL(k)$ nous devrions donc modifier celles qui ne le sont pas pour effectuer l'analyse.

1.2 Analyse Syntaxique et approche ascendante (Grammaires $LR(k)$)

1.2.1 Principe

Une autre approche de l'analyse syntaxique grâce aux grammaires et d'utiliser l'approche *descendante*. En effet, depuis le début nous cherchons à dériver un mot à partir de l'axiome de départ en descendant vers des états terminaux. Nous allons ici chercher à faire le contraire : à partir d'un mot nous chercherons à le réduire vers l'axiome. C'est ce que l'on appelle l'approche *ascendante*, aussi appelée *analyse par décalage/réduction* (*shift-reduce*).

Exemple Commençons par étudier un exemple pour bien comprendre le principe de ces grammaires. Soit la grammaire G_1 suivante :

$$\begin{cases} E \rightarrow E + E \\ E \rightarrow E * E \\ E \rightarrow (E) \\ E \rightarrow id \end{cases}$$

Soit $w \in \Sigma^*$ tel que $w := id + id * id$. Nous cherchons à réduire w grâce aux règles de productions de G_1 jusqu'à l'axiome S . Résumons tout ça dans un tableau :

Proto-phrase de droite	Poignée (handle)	Production de réduction
$id + id * id$	id	$E \rightarrow id$
$E + id * id$	id	$E \rightarrow id$
$E + E * id$	id	$E \rightarrow id$
$E + E * E$	$E * E$	$E \rightarrow E * E$
$E + E$	$E + E$	$E \rightarrow E + E$
E		

1.2.2 Outils Fondamentaux

Définissons maintenant quelques outils pour effectuer rigoureusement cette analyse ascendante.

Définition (proto-mot) . Soit $G = (V, \Sigma, S, P)$ une grammaire. Un *proto-mot* (ou *chaîne sententielle*) est toute chaîne de symboles, terminaux et/ou non terminaux, que l'on peut obtenir en partant de l'axiome et en effectuant un nombre quelconque de dérivations. For-

mellement, pour la grammaire G , on a :

$$\alpha \in (V \cup \Sigma)^* \text{ est un proto-mot si } S \rightarrow^* \alpha$$

Un proto-mot est simplement une "étape intermédiaire" de la dérivation d'un mot. Dans l'ensemble précédent, cela correspond à $E + E * id$. Il est composé du terminal id et du non terminal E .

Définition (Poignée (Handle)) . Soient $G = (V, \Sigma, S, P)$ une grammaire et w un proto-mot de G . Une *poignée* de w est occurrence précise du corps d'une production $A \rightarrow \alpha$ dans w telle que :

$$S \rightarrow^* \gamma A \delta \rightarrow \gamma \alpha \delta = w$$

Une poignée correspond donc à une sous-chaîne à réduire dans notre mot w à un instant donné de l'analyse ascendante. Dans notre exemple précédent, pour le proto-mot $E + E * id$, une poignée peut correspondre au non terminal id qui se réduit en E en une étape grâce à la règle $E \rightarrow id$.

Définition (Préfixe Viable) . Soient $G = (V, \Sigma, S, P)$ une grammaire et w un proto-mot de G . Une *préfixe viable* est toute chaîne de symbole $\alpha \in V^*$ telle que :

$$\exists \beta \gamma \text{ tels que } S' \rightarrow^* \alpha \beta \gamma \text{ et } \beta \gamma \text{ contient une poignée complète}$$

où $S' \rightarrow S$ est l'axiome augmenté.

Un préfixe viable est exactement ce que l'on veut avoir au dessus de la pile d'un analyseur LR si l'on ne veut pas de retrouver dans une impasse :

- Si la pile contient un préfixe viable, on est dans une configuration valide et l'analyse peut continuer.
- Si la pile contient quelque chose qui n'est pas un préfixe viable, alors l'analyseur détecte une erreur syntaxique.

Définition (Configuration Valide) . Soient $G = (V, \Sigma, S, P)$ une grammaire. Une configuration d'un analyseur ascendant se décrit par :

- Le *contenu de la pile*
- Le *reste de l'entrée* non consommé.

Une configuration est dite *valide* si la pile contient un *préfixe viable* qui peut mener à une dérivation complète de l'axiome.

Une configuration valide est une sorte d'instantané de l'analyse où l'on n'a pas fait d'erreur de parsing.

1.2.3 Analyseur LR(k)

Cette ensemble de définitions nous permet de définir formellement les grammaires $LR(k)$ de la façon suivante.

Définition (Grammaire $LR(k)$) . Soit $G = (V, \Sigma, P, S)$ une grammaire, où :

- V est l'ensemble des symboles (terminaux et non-terminaux),
- $\Sigma \subseteq V$ est l'ensemble des terminaux,
- P est l'ensemble des productions de la forme $A \rightarrow \alpha$,
- $S \in V \setminus \Sigma$ est l'axiome.

On dit que G est une *grammaire LR(k)* si et seulement si, pour toute dérivation à droite :

$$S \Rightarrow^* \gamma A \omega \Rightarrow \gamma \beta \omega$$

avec $A \rightarrow \beta \in P$, $\gamma, \omega \in V^*$, il existe un *unique* choix de production $A \rightarrow \beta$ qui peut être appliqué pour réduire la *poignée* β , en ne regardant que :

- le *préfixe viable* $\gamma\beta$ (ce qui est déjà sur la pile),
- et au plus k symboles de lookahead dans ω .

En d'autres termes, la suite β est **une manche unique et non ambiguë** que l'analyseur peut identifier avec seulement k symboles d'anticipation.

Remarque On parlera identiquement de *grammaire LR(k)* ou d'*analyseur LR(k)*. Le terme *LR(k)* signifie :

- **Left to right** : l'analyseur lit les mots de gauche à droite.
- **Rightmost derivation** : les motifs spécifiés par la grammaire sont recherchés par suffixes de la chaîne lue.
- k : correspond au nombre de lettres non consommées considérées pour le choix de la dérivation à choisir.

Définissons maintenant un automate capable de reconnaître de telles grammaires.

Définition (Automate LR (bottom-up)) . Un *automate LR* est un *automate à pile* qui reconnaît les préfixes viables d'une grammaire hors contexte $G = (V, \Sigma, S, P)$ où :

- la *pile* contient les symboles et états représentant un *préfixe viable*.
- l'entrée est la chaîne de symboles à analyser terminant par le caractère de fin \$.

L'objectif est de lire la chaîne de gauche à droite en réduisant progressivement toutes poignées dans le but d'obtenir l'axiome sur le sommet de la pile et d'avoir consommé toute l'entrée.

Formellement, l'automate peut être défini par :

$$M = (Q, \Sigma, \Gamma, ACTION, GOTO, q_0, Z_0, F)$$

où :

- Q est l'ensemble des états de l'automate.
- Σ représente l'alphabet terminal (lettres).
- $\Gamma = \Sigma \cup V_N$ est l'ensemble des symboles de la pile.
- q_0 est l'état initial.
- Z_0 est le symbole initial de la pile.
- F est l'ensemble des états acceptants.

On définit en plus deux tables qui définissent la dynamique de l'analyseur :

- La table *ACTION* qui détermine quelle action effectuer à la lecture d'une lettre :

$$ACTION : Q \times (\Sigma \cup \{\$\}) \longrightarrow \{shift(q), reduce(A \rightarrow \beta), accept, error\}$$

Elle gère les transitions sur les terminaux.

- La table *GOTO* qui détermine l'état dans lequel doit se trouver l'automate après une réduction :

$$\boxed{GOTO : Q \times V \longrightarrow Q}$$

Elle gère les transitions sur les non terminaux.

Définition (Shift et Reduce) . Comme dit précédemment, nous n'avons que deux opérations à effectuer lors de la lecture d'un mot par une grammaire $LR(k)$ (ou un automate LR) : *shift* ou *reduce*. Définissons les proprement :

1. L'opération *shift* consiste à déplacer le prochain symbole à lire sur la pile et aller à l'état correspondant. On avance donc dans la lecture mais sans faire de réduction.
2. L'opération *reduce* consiste, lorsque le sommet correspond à une poignée β pour une règle de production $A \rightarrow \beta$, de :
 - Dépiler $|\beta|$ symboles et états
 - Empiler le non-terminal A
 - Aller dans l'état déterminé par la table *GOTO*
3. *Accept* : si la pile est seulement constituée de S' et que l'entrée est \$, alors le mot est reconnu.
4. *Error* : si aucune opération shift/reduce n'est possible alors la configuration est invalide et le mot n'est pas reconnu.

Nous savons maintenant analyser un mot par un automate mais il reste un problème : comment définir les états de l'automate ? Pour cela, nous allons avoir besoin de définir le concept de *fermeture* ou *clôture*.

Définition (Fermeture) . Soit $G' = \{V, \Sigma, P, S'\}$ une grammaire augmentée. Un *item* $LR(0)$ est une règle de la forme :

$$A \rightarrow \alpha \cdot \beta$$

où "." indique la position de la l'analyse dans la production $A \rightarrow \alpha\beta$. La *fermeture* d'un ensemble d'items I , notée $cl(I)$ est définie récursivement ainsi :

1. $\forall \text{ item} \in I, \text{ item} \in cl(I)$.
2. Pour chaque item $A \rightarrow \alpha \cdot B\beta \in cl(I)$, et pour chaque production $B \rightarrow \gamma \in P$, alors $B \rightarrow \cdot \mu \in cl(I)$.

Exemple (Calcul de fermeture) Soit G une grammaire dont les règles de production sont :

$$\begin{cases} S' \rightarrow S \\ S \rightarrow AA \\ A \rightarrow a \end{cases}$$

On pose :

$$I_0 := \{S' \rightarrow \dots S\}$$

Calculons maintenant les fermetures :

- Le point est avant S donc on ajoute $S \rightarrow \cdot AA$ grâce à la règle $S \rightarrow AA$.
- Récursivement, le point est avant A donc on ajoute $A \rightarrow \cdot a$ grâce à la règle $A \rightarrow a$.
- Le point $a \in \Sigma$ est un terminal on s'arrête donc ici.

On a donc :

$$cl(I_0) = \{S' \rightarrow \cdot S, S \rightarrow \cdot AA, A \rightarrow \cdot a\}$$

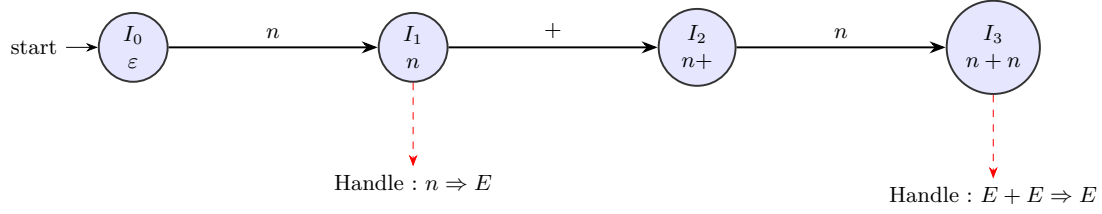


FIGURE 1.1 – Préfixes viables et leur correspondance avec les états de l'automate LR. Chaque état I_i représente un ensemble de *préfixes viables* atteignables. Les flèches indiquent les transitions de lecture (shift). Les lignes rouges en pointillés indiquent les poignées que l'on peut réduire à partir de cet état.

Exemple Travaillons autour d'un exemple pour bien saisir toutes ces nouvelles notions. Soit G une grammaire dont les règles de production sont :

$$\begin{cases} S' \rightarrow S \\ S \rightarrow AA \\ A \rightarrow a \end{cases}$$

On a ajouté un symbole S' fictif pour avoir un axiome de départ unique. Comme vu précédemment, commençons par calculer les clôtures :

- On calcule la clôture de I_0 qui donne :

$$cl(I_0) = \{S' \rightarrow \cdot S, S \rightarrow \cdot AA, A \rightarrow \cdot a\}$$

- Après la lecture de S nous devons passer dans un nouvel état I_1 d'où :

$$cl(I_1) = \{S' \rightarrow S \cdot\}$$

- Après la lecture de A dans I_0 on obtient l'ensemble I_2 où :

$$cl(I_2) = \{S \rightarrow A \cdot A, A \rightarrow \cdot a\}$$

- Après la lecture de A depuis I_2 , on obtient I_3 :

$$cl(I_3) = \{S \rightarrow AA \cdot\}$$

- Pour finir, après la lecture de a depuis I_0 ou I_2 , on a :

$$cl(I_4) = \{A \rightarrow \cdot a\}$$

On a donc les transitions suivantes :

$$\text{GOTO}(I_0, S) = I_1, \quad \text{GOTO}(I_0, A) = I_2, \quad \text{GOTO}(I_0, a) = I_5$$

$$\text{GOTO}(I_2, A) = I_4, \quad \text{GOTO}(I_2, a) = I_4$$

On peut donc construire la table action (transitions sur les terminaux) :

État	a	$\$$
I_0	shift I_4	error
I_1	error	accept
I_2	shift I_4	error
I_3	reduce $S \rightarrow AA$	reduce $S \rightarrow AA$
I_4	reduce $S \rightarrow A$	reduce $S \rightarrow A$

Et la table goto sur les non terminaux :

État	S	A
I_0	I_1	I_2
I_1	—	—
I_2	—	I_3
I_3	—	—
I_4	—	—

Résumons le traitement de l'entrée $w = a a \$$:

Pile (état/symbole)	Entrée	Action	Commentaire
I_0	a a \$	s I_4	Décalage sur a, empiler I_4
I_0 a I_4	a \$	r($A \rightarrow a$)	Réduction : dépiler a et I_4 , empiler A puis GOTO[I_0 ,A]= I_2
I_0 A I_2	a \$	s I_4	Décalage sur a, empiler I_4
I_0 A I_2 a I_4	\$	r($A \rightarrow a$)	Réduction : dépiler a et I_4 , empiler A puis GOTO[I_2 ,A]= I_3
I_0 A I_2 A I_3	\$	r($S \rightarrow AA$)	Réduction : dépiler deux A et états, empiler S puis GOTO[I_0 ,S]= I_1
I_0 S I_1	\$	accept	Mot reconnu, analyse terminée

